DOCUMENTATION TECHNIQUE

Application Météo 7 jours

Présentation générale

L'application Météo 7 jours est une application web développée dans le cadre du BTS SIO option SLAM. Elle permet de consulter les prévisions météorologiques sur une période de sept jours pour une ville française. L'application repose sur l'exploitation de services externes accessibles via API et fonctionne entièrement côté client dans le navigateur.
L'objectif technique du projet est de démontrer l'utilisation d'API REST, la manipulation de données JSON et l'affichage dynamique de données dans une interface web interactive.

Architecture de l'application

L'application repose sur une architecture ***frontend orientée client,*** où l'ensemble de la logique applicative est exécuté dans le navigateur de l'utilisateur.

Le fonctionnement général est le suivant :

1. L'utilisateur saisit une ville ou un code postal dans l'interface.
2. Une requête HTTP est envoyée à ***l'API Nominatim*** afin d'obtenir les coordonnées géographiques correspondantes.
3. Les coordonnées récupérées sont utilisées pour interroger l'***API Open-Meteo***.
4. Les données météorologiques sont récupérées au ***format JSON***.
5. Les données sont traitées en JavaScript puis affichées dynamiquement dans l'interface.

L'application ne nécessite pas de serveur backend. Les appels API sont réalisés directement depuis le navigateur à l'aide de requêtes HTTP.

Schéma d'architecture de l'application

L'application suit une ***architecture web simple*** reposant sur l'interaction entre le navigateur et des services externes.
Le navigateur exécute le code JavaScript qui envoie des requêtes HTTP vers les API externes afin de récupérer les données nécessaires à l'affichage des prévisions météorologiques.
(Cf Figure 1 : Diagramme de processus d'application météo)

Cette architecture permet de créer une application légère ne nécessitant pas d'infrastructure serveur.

Technologies et services utilisés

Frontend

HTML5
HTML est utilisé pour structurer l'interface de l'application. Il définit les différentes zones de l'application telles que la barre de recherche, l'affichage des prévisions météo, les graphiques et la carte interactive.

CSS3
CSS est utilisé pour la mise en forme de l'interface utilisateur et l'amélioration de l'expérience visuelle.

Bootstrap 5
Bootstrap est utilisé afin de faciliter la création d'une interface responsive et mobile-first. Il permet d'adapter l'affichage de l'application aux différents types d'écrans.

JavaScript (ES6)
JavaScript constitue la logique principale de l'application. Il est utilisé pour gérer les interactions utilisateur, effectuer les appels API, traiter les réponses JSON et mettre à jour dynamiquement l'interface.

Bibliothèques JavaScript

Chart.js
Chart.js est utilisé pour représenter les données météorologiques sous forme de graphiques interactifs. Ces graphiques permettent d'observer l'évolution de différentes données comme la température, l'humidité, le vent et les précipitations.

Leaflet.js
Leaflet est utilisé pour afficher une carte interactive dans l'application. La carte permet de localiser la ville recherchée à partir de ses coordonnées géographiques.

OpenStreetMap
OpenStreetMap fournit les données cartographiques utilisées par Leaflet pour afficher la carte.

APIs utilisées

API Open-Meteo
L'API Open-Meteo fournit les prévisions météorologiques pour les coordonnées géographiques obtenues.

Les données récupérées incluent notamment :

- température minimale et maximale
- humidité
- vitesse du vent
- précipitations

- heures de lever et de coucher du soleil

Les données sont retournées au format JSON et sont ensuite traitées en JavaScript afin d'être affichées dans l'interface.

API Nominatim

L'API Nominatim est utilisée pour effectuer le géocodage. Elle permet de convertir un nom de ville ou un code postal en coordonnées géographiques (latitude et longitude).
Ces coordonnées sont nécessaires pour effectuer les requêtes vers l'API météo.

Flux des API

Le fonctionnement de l'application repose sur plusieurs échanges entre le navigateur et les API externes. (C*f. Figure 2 : Diagramme du flux des API*)

Étape 1 : saisie de la ville

L'utilisateur saisit le nom d'une ville ou un code postal dans le champ de recherche de l'application puis lance la recherche.

Étape 2 : requête vers l'API Nominatim

L'application envoie une requête HTTP vers l'***API Nominatim*** afin d'obtenir les coordonnées géographiques de la ville saisie.

Étape 3 : récupération des coordonnées

L'application analyse la réponse JSON et récupère les coordonnées géographiques nécessaires pour interroger l'API météo.

Étape 4 : requête vers l'API Open-Meteo

Une seconde requête HTTP est envoyée vers l'***API Open-Meteo*** en utilisant les coordonnées obtenues afin de récupérer les prévisions météorologiques.

Étape 5 : traitement des données

Les données retournées au format JSON sont analysées par le code JavaScript afin d'extraire les informations nécessaires.

Étape 6 : affichage dans l'interface

Les données sont ensuite affichées dans l'application sous différentes formes :

- cartes météo journalières
- graphiques interactifs avec Chart.js
- statistiques moyennes
- carte interactive avec Leaflet

Fonctionnement de l'application

Recherche de ville

Lorsque l'utilisateur saisit une ville dans le formulaire de recherche, la fonction ***handleSearch()*** est exécutée. Cette fonction récupère la valeur saisie dans le champ de recherche.

Une requête HTTP est ensuite envoyée vers l'***API Nominatim*** afin d'obtenir les coordonnées géographiques correspondant à la ville.

Récupération des données météo

Une fois les coordonnées obtenues, une seconde requête HTTP est envoyée vers l'***API Open-Meteo***. La réponse contient les données météorologiques au format JSON qui sont ensuite analysées par le code JavaScript.

Traitement et affichage des données

Les données météo récupérées sont utilisées pour générer dynamiquement plusieurs éléments de l'interface :

- cartes météo journalières
- statistiques calculées à partir des données
- graphiques interactifs
- carte de localisation

Les éléments HTML sont créés ou mis à jour dynamiquement grâce à JavaScript.

Affichage de la carte

Les coordonnées de la ville sont utilisées pour initialiser une ***carte Leaflet*** centrée sur la localisation correspondante. Un marqueur est placé sur la carte afin d'indiquer la position de la ville recherchée.

Gestion des erreurs

L'application intègre un système de gestion des erreurs afin d'assurer un fonctionnement stable.

Plusieurs situations sont prises en compte :

- ville non trouvée par l'API Nominatim
- absence de données météo
- erreur lors des appels API
- problème de connexion Internet

Dans ces cas, un message est affiché dans l'interface afin d'informer l'utilisateur.

Structure du projet

Le projet est organisé de manière simple afin de faciliter sa compréhension et sa maintenance.

Les principaux éléments du projet sont :

- un fichier HTML principal contenant la structure de l'application
- des fichiers CSS contenant les styles de l'interface
- un fichier JavaScript contenant la logique de l'application
- l'intégration de bibliothèques externes pour les graphiques et la carte

Environnement de développement

Le projet a été développé à l'aide des outils suivants :

Visual Studio Code
VS Code est utilisé comme environnement de développement principal.

Live Server
L'extension Live Server permet de lancer un serveur local afin de tester l'application dans un navigateur pendant le développement.

Git et GitHub
Git est utilisé pour le versionnement du code. Le projet est hébergé sur GitHub afin de faciliter le partage et la gestion des versions.

Conclusion

L'application Météo 7 jours illustre l'utilisation d'API REST dans une application web moderne. Elle démontre la capacité à récupérer des données externes, les traiter au format JSON et les afficher dynamiquement dans une interface interactive.

Le projet met en pratique plusieurs compétences importantes du développement web, notamment l'exploitation d'API, la manipulation de données JSON, l'utilisation de bibliothèques JavaScript et la conception d'une interface web dynamique.

Annexes

Figure 1: Diagramme de processus d'application météo
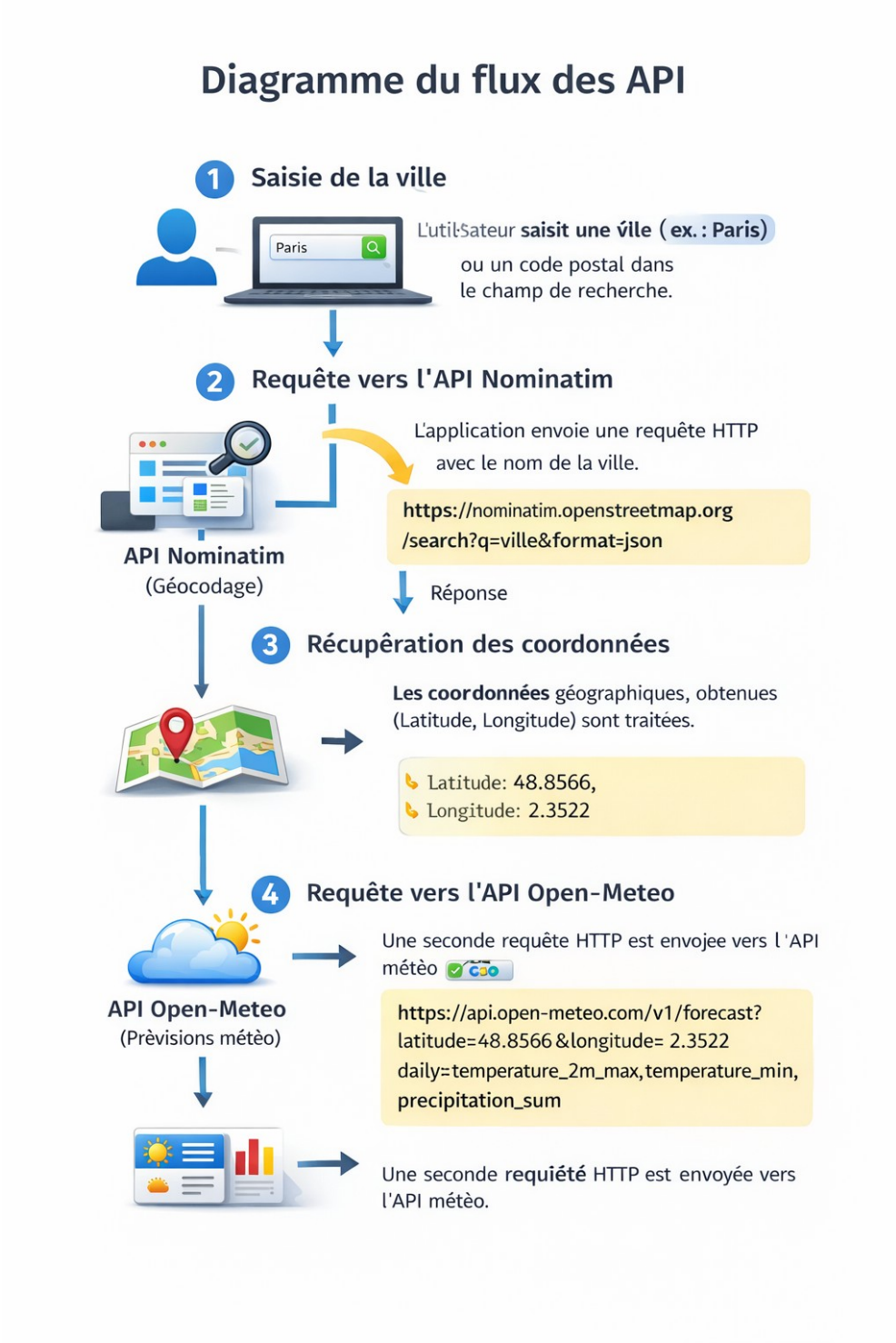
Figure 2: Diagramme du flux d'API météo